



26

Sécurité & menaces sur l'IA

Sécurité · C2

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Construire un threat model (STRIDE / attack tree)
- › Prioriser 5 contrôles : 4 prouvés + audit Planifié v0
- › Prouver auth, validation, débit et taille de payload
- › Spécifier l'audit futur et les risques résiduels

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE threat_model.md · registre 4/1 · suites API/sécu à 0 échec

1

Module 26 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Quelqu'un sur le réseau bombarde ton API de requêtes et tente d'envoyer un fichier de 2 Go. Sans garde-fous, le service tombe — et la chaîne de production avec.

Angle éthique / risque (C2) C'est LE module C2 : clé d'API, limite de débit et taille de payload protègent les données et la disponibilité. On pense la menace AVANT l'incident (STRIDE).

■ Contexte & outils — pourquoi ce module

Contexte : une API ouverte est une cible. Pourquoi : on la protège et on anticipe les menaces spécifiques à l'IA.

- Clé d'API (X-API-Key) : sans clé -> 401 refusé
- Rate limit : trop de requêtes -> 429 (anti-abus)
- Limite de taille : payload trop gros -> 413
- STRIDE : méthode pour lister les menaces d'un système
- Garde-fous : validation stricte + logs sans secret

Objectif du module Une API qui refuse proprement les abus, sans jamais fuiter de secret.

■ Penser comme un attaquant : STRIDE

Avant de coder des défenses, on cartographie les menaces avec une méthode : STRIDE.

- S poofing (usurpation) · T ampering (altération) · R epudiation (déni d'action)
- I nformation disclosure (fuite) · D enial of service · E levation of privilege
- On l'applique à l'API ET au pipeline de données (arbres d'attaque)

i Le jargon décodé

Threat model = modèle de menaces. Attack tree = arbre des chemins d'attaque. Surface d'attaque = tout ce qui est exposé.

≈ L'analogie

Modéliser les menaces, c'est faire le tour de la maison avant d'acheter des serrures : inutile de blinder la porte si la fenêtre est ouverte.

EN UNE PHRASE On cartographie ce qui peut mal tourner AVANT de coder les défenses.

■ Les menaces propres à l'IA

Un modèle ML ajoute des vulnérabilités que le logiciel classique n'a pas.

- Adversarial : entrées modifiées pour tromper le modèle
- Vol de modèle : le reconstituer par requêtes massives
- Empoisonnement : corrompre les données d'entraînement
- Fuite : le modèle ou les logs révèlent des données sensibles

? Le saviez-vous ?

En adversarial, changer quelques pixels invisibles à l'œil peut faire prendre un « stop » pour une limitation de vitesse par un modèle de vision.

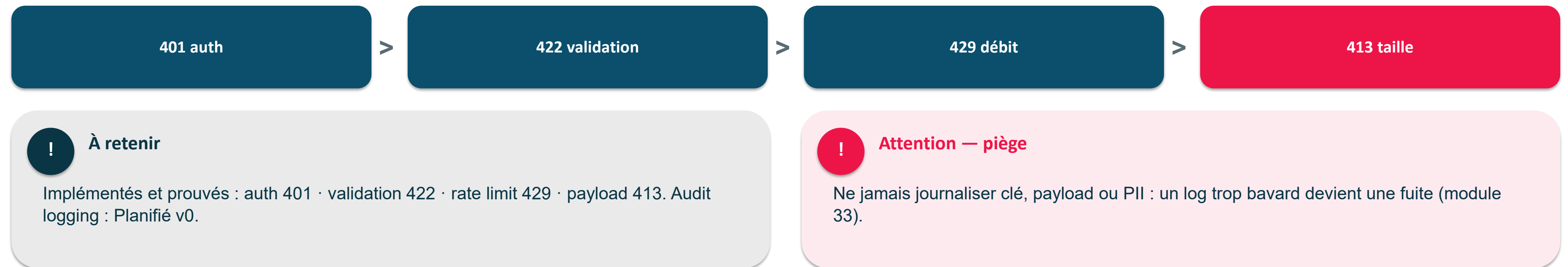
→ Cas réel InduSense

Un attaquant qui spamme /predict peut faire tomber le service (DoS) ET voler le modèle par extraction → rate limit + auth.

EN UNE PHRASE Aux menaces web classiques s'ajoutent les menaces spécifiques au modèle.

■ 5 contrôles prioritaires : 4 prouvés + 1 planifié

Un contrôle n'est acquis que si son implémentation et sa preuve existent.



EN UNE PHRASE Priorisé ne veut pas dire implémenté : quatre preuves, une preuve future.

■ Les 4 contrôles prouvés, dans le code

Security.py, main.py, les schémas et les tests prouvent quatre défenses ; l'audit reste absent.

```
MAX_BODY_BYTES = 64 * 1024          # > 64 Ko -> 413
def rate_limit_dependency(request):   # politique 60/60 -> 429
def require_api_key(...):             # absente/invalid -> 401
# validation Pydantic v2 -> 422
# audit logging -> Planifié v0 (pas de middleware/test)
```

! Attention — piège

Le request-id corrèle ; il n'est pas un audit log. Ne pas utiliser Depends(rate_limit) : limit/window seraient exposés.

→ Cas réel InduSense

STRIDE sur /predict-tabular : usurpation -> clé ; DoS -> rate limit ; fuite -> pas de secret en log ; altération -> Pydantic.

EN UNE PHRASE Quatre contrôles prouvés ; audit logging Planifié v0.

■ À retenir en 3 points

- Pas de clé → 401 ; trop de requêtes → 429 ; payload trop gros → 413.
- La sécurité, ça se teste comme le reste du code.
- On modélise la menace (STRIDE) avant de coder la parade.

2

Module 26 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — attack tree & contrôles

```
# preuves attendues  
60 appels acceptés, 61e -> 429 · payload > 64 Ko -> 413  
suites API + sécurité -> 0 échec
```

✅ Preuve attendue threat_model.md + registre 4 Implémenté / 1 Planifié v0 + 401/422/429/413 verts.

■ TP — durcir sans casser

- Brancher rate_limit_dependency (le wrapper fermé) et limit_body_size sur /predict-tabular
- Vérifier que /health reste libre (liveness)

Lien module 33 Ne jamais mettre clé / payload dans les logs ou labels → fuite.

3

Module 26 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — étendre la sécurité

Avancez à votre rythme. Le formateur reste disponible.

- Tester que l'OpenAPI masque limit/window via rate_limit_dependency
- OU tester la non-divulgence clé / payload / PII dans les logs

+ Preuve, pas promesse

La non-divulgence prouve l'absence de fuite, pas l'existence d'un audit log : il reste Planifié v0.

■ Definition of Done & transition

- DoD : threat model + registre 4 Implémenté / 1 Planifié v0 + suites API/sécu à 0 échec

Transition → Module 27 Service durci → on l'emballage dans une image Docker reproductible.